

Android Security Assessment

R2Pay v1.0

1. Introduction

Target: R2Pay v1.0 APK

Platform: Android

Analysis Type: Static Analysis, Smali Analysis, APK Modification, Dynamic Analysis

This security assessment was conducted against the R2Pay v1.0 Android application in a controlled testing environment. The assessment focused on analyzing client-side security mechanisms implemented by the application, particularly root detection, anti-instrumentation mechanisms, and the application's token-generation flow. The assessment objectives were:

- Identifying root detection mechanisms implemented by the application.
- Analyzing the application's RootBeer implementation.
- Analyzing security-sensitive application logic through Smali code.
- Modifying the application's Smali code to bypass identified client-side security checks.
- Rebuilding and signing the modified APK.
- Installing and validating the modified APK in an Android x86_64 emulator environment.
- Identifying the PIN, amount, and token-generation flow.
- Investigating anti-Frida protection.

2. Overview

2.1 Environment Setup

Test Environment:

- Application: R2Pay v1.0
- Android Environment: LDPlayer
- Architecture: x86_64
- Host Environment: Kali Linux

The R2Pay application was executed inside an isolated LDPlayer Android environment. The emulator environment contained root-related artifacts, including:

- /system/bin/su
- /system/xbin/su

These artifacts were subsequently detected by the application's RootBeer implementation. The application was initially unable to start because the root-detection result caused an intentional exception inside MainActivity.onCreate().

2.2 Tools

Tool	Purpose
JADX	Java/decompiled code analysis
Apktool	APK decoding, Smali analysis, rebuilding
apksigner	APK signing and signature verification
ADB	Device communication, application installation and Logcat analysis
LDPlayer	Android x86_64 testing environment
Frida	Dynamic instrumentation testing
Logcat	Runtime behavior and exception analysis

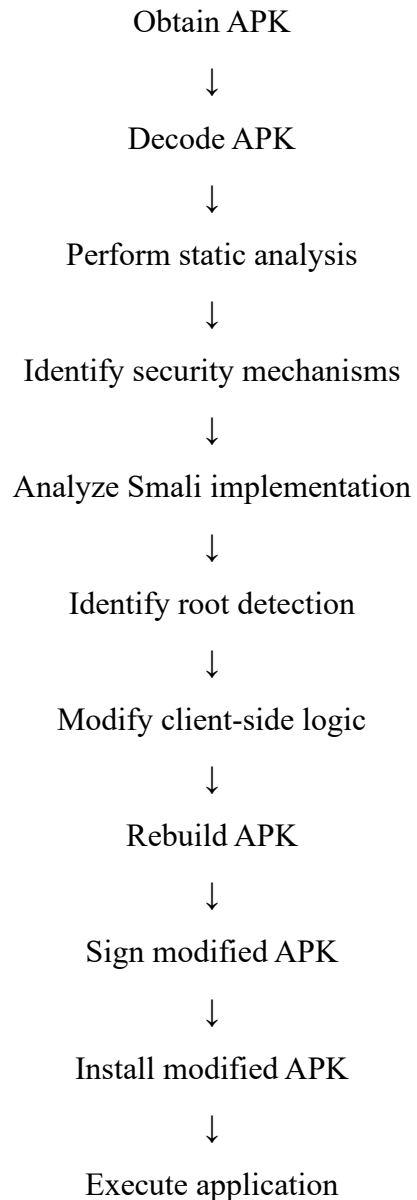
3. Findings

3.1 Findings Summary

Finding	Severity	Result
Client-side root detection can be bypassed	Medium	Bypassed
Root detection is implemented at multiple execution points	Medium	Identified
Token generation logic is exposed in the client	High	Partially analyzed
Anti-Frida / anti-instrumentation protection	Medium/High	Unresolved
PIN / amount / token validation	High	Partially analyzed / unresolved

3.2 Security Impact

The primary security concern identified during the assessment is the reliance on client-side security controls. The application implements security-sensitive decisions inside the Android client, including root detection and logic associated with token generation. The general attack path can be represented as:



The assessment demonstrated that at least some client-side security controls could be modified without requiring changes to the original application source code.

4. Detail Findings

4.1 Initial Static Analysis

The initial APK analysis was performed using JADX and Apktool. The analysis focused on identifying the main application components and locating security-sensitive functionality. The MainActivity class was identified as one of the primary components responsible for:

- Application initialization.
- Root detection.
- User input processing.
- PIN validation.
- Amount validation.
- Token-generation flow.

4.2 Root Detection Analysis

During analysis of MainActivity.onCreate(), an object corresponding to the application's RootBeer implementation was identified:

```
new-instance v0, L♫/u;
```

```
invoke-direct {v0, v1}, L♫/u;.<init>(Landroid/content/Context;)V
```

The application subsequently invokes multiple methods:

```
invoke-virtual {v0}, L♫/u;.<f>()Z
```

```
invoke-virtual {v0}, L♫/u;.<θ>()Z
```

```
invoke-virtual {v0}, L♫/u;.<ö>()Z
```

The returned values determine whether execution enters the protected branch. The relevant control flow eventually reaches:

```
:cond_0
```

```
const/16 v1, 0x539
```

div-int/lit8 v1, v1, 0x0

The instruction intentionally performs an integer division by zero. This produces:
java.lang.ArithmeticException: divide by zero

4.3 Root Detection Runtime Evidence

Runtime analysis using Logcat provided direct evidence that RootBeer detected root-related binaries in the testing environment. The application reported:

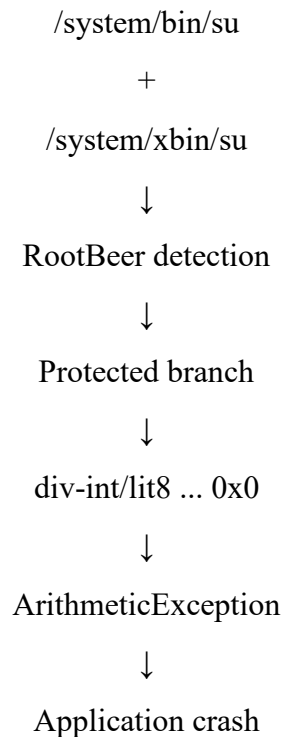
RootBeer: 0() [194] - /system/bin/su binary detected!

RootBeer: 0() [194] - /system/xbin/su binary detected!

Immediately afterwards, the Android runtime terminated the application:

- FATAL EXCEPTION: main
- java.lang.RuntimeException: Unable to start activity
- java.lang.ArithmeticException: divide by zero

The observed execution flow was therefore:



4.4 Root Detection Bypass

The root-detection section of onCreate() was removed from the locally modified APK.

The patched implementation retains the normal initialization flow:

- input-byte v0, p0, Lre/pwnme/MainActivity;->0:B
- invoke-virtual {p0}, Lre/pwnme/MainActivity;->W()V
- return-void

The modification removes the RootBeer initialization and the subsequent conditional branch responsible for reaching the divide-by-zero exception. The modified application was subsequently rebuilt and executed in LDPlayer.

Result Status: Bypassed

4.5 Secondary Root Detection

An important observation was made during analysis of the application's onClick() method. The application performs another RootBeer check during the token-generation process. After constructing the input buffer, the application creates another RootBeer instance:

- new-instance v2, L\$u;

It then executes:

- invoke-virtual {v2}, L\$u;->f()Z
- invoke-virtual {v2}, L\$u;->0()Z
- invoke-virtual {v2}, L\$u;->ö()Z

If the detection condition is triggered, the application enters another protected branch. The branch modifies an internal state and subsequently executes:

- new-instance v7, Ljava/lang/NullPointerException;
- invoke-direct {v7}, Ljava/lang/NullPointerException;-><init>()V
- invoke-virtual {v7}, Ljava/lang/Object;->notify()V

This indicates that root detection is not limited to application startup. The application also performs root-related checks before or during the tokenization process.

Result Status: Identified

This finding demonstrates that multiple execution points should be analyzed when assessing client-side root protection.

4.6 PIN Validation

The token-generation process begins with validation of the PIN input.

The application retrieves the PIN value and checks its length:

- `invoke-virtual {v2}, Ljava/lang/String;.>length()I`
- `const/4 v7, 0x4`
- `if-eq v2, v7, :cond_0`

The application therefore expects the PIN to contain exactly four characters. If the condition is not satisfied, the application displays:

- PIN code must be 4 digits long...

The PIN is subsequently converted into an integer and formatted using:

- `%08d`

This indicates that the original PIN is transformed before being used in the token-generation process.

Status: Partially analyzed

The PIN-processing stage has been identified, but the complete security mechanism responsible for validating or transforming the PIN into the final token has not been fully recovered.

4.7 Amount Processing

The application also processes the amount entered by the user.

The amount is retrieved and its length is checked:

- `invoke-virtual {v2}, Ljava/lang/String;.>length()I`
- `const/16 v7, 0x8`

- if-gt v2, v7, :cond_4

Therefore, the application limits the amount input to a maximum of eight characters. The amount is then converted to an integer and formatted using:

- %08d

If the amount is larger than the accepted length, the application displays: Enormous amount to tokenize r2coins. Radare2 is a non-profit organization :)

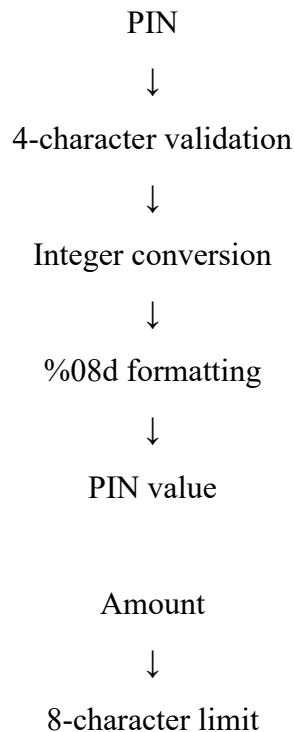
The processing logic demonstrates that both PIN and amount undergo client-side transformation before token generation.

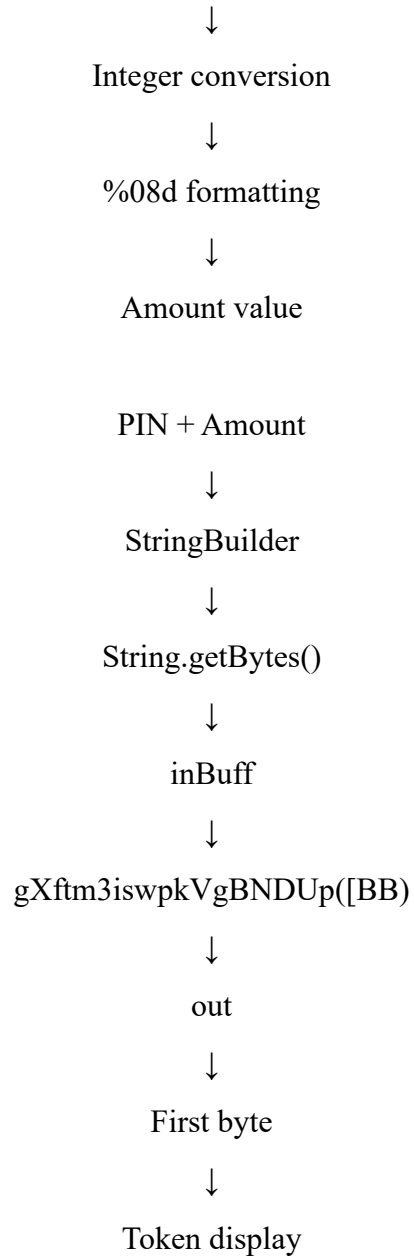
Status: Partially analyzed

The input validation and formatting mechanism was identified, but complete recovery of the security algorithm was not achieved.

4.8 Token Generation Flow

After PIN and amount processing, the application concatenates the transformed values. The relevant flow can be represented as:





The source shows the concatenated PIN and amount being converted into a byte array. The byte array is then passed to:

- invoke-virtual {v7, v0, v8},
- Lre/pwnme/MainActivity;->gXftm3iswpkVgBNDUp([BB)[B

The returned byte array is subsequently processed to produce the displayed token.

Status: Pipeline identified

The token-generation pipeline was successfully identified; however, the complete implementation of the underlying algorithm was not recovered.

4.9 Anti-Frida / Runtime Instrumentation Analysis

4.9.1 Testing Approach

The test was performed against the re.pwnme application running on an x86_64 Android emulator. Frida was used to instrument native functions from the application's process. Two native functions were targeted:

- `open()` — to observe file paths accessed by the application.
- `snprintf()` — to observe and modify strings generated during native execution.

The following Frida script was used during the test in the repository at `evidence/frida-script.js`

The script was adapted from an external Android/Frida reverse-engineering tutorial. The original video reference is: [Android Frida Reverse Engineering Tutorial — YouTube](#)

4.9.2 Runtime Observation

The `open()` hook successfully captured multiple file-access operations performed by the application. Among the observed paths were:

- Open: `/data/local/su`
- Open: `/data/local/bin/su`
- Open: `/data/local/sbin/su`
- Open: `/sbin/su`
- Open: `/su/bin/su`
- Open: `/system/bin/su`
- Open: `/system/bin/.ext/su`
- Open: `/system/bin/failsafe/su`
- Open: `/system/sd/sbin/su`
- Open: `/system/usr/we-need-root/su`
- Open: `/system/sbin/su`
- Open: `/cache/su`

- Open: /data/su
- Open: /dev/su

This is significant because it provides runtime evidence that the native component accesses multiple well-known su paths.

The application also loaded its native library:

Open: /data/app/re.pwnme-5PODr-Z6llc_b3rj8mEf0w==/lib/x86_64/libnative-lib.so

The snprintf() hook also produced output associated with /status and /proc/self/fd/:

[snprintf] /fake_status

...

[snprintf] /proc/self/fd/42

This indicates that the Frida hooks were successfully executed before the application terminated.

4.9.3 Process Crash

Although the hooks were successfully installed and generated runtime output, the application subsequently crashed:

- Process crashed: Bad access due to invalid address
- The backtrace identified the native library as the crash location: #00 pc 000000000011c58e
- /data/app/re.pwnme-5PODr-Z6llc_b3rj8mEf0w==/lib/x86_64/libnative-lib.so

The crash therefore occurred inside the application's native component rather than providing a successful runtime bypass. At this stage, the available evidence is insufficient to conclude that the crash was directly caused by an anti-Frida mechanism.

Possible explanations include:

- incompatibility between the hook and the target native implementation;
- incorrect assumptions about the snprintf() argument layout;
- interaction between Frida and the application's native protection mechanism;

4.9.4 Analysis

The test produced partial evidence of successful dynamic instrumentation. The following activities were successfully demonstrated:

Test	Result
Attach Frida to application	Successful
Hook native open()	Successful
Observe native file access	Successful
Identify su path checks	Successful
Hook native snprintf()	Successful / Partially successful
Modify native string data	Attempted
Maintain application stability	Failed
Confirm anti-Frida protection	Unresolved
Achieve root-detection bypass	Unresolved

The most important observation is that the application performs native access to multiple su paths, while the Java/Smali analysis had already identified RootBeer-related root detection.

Therefore, the runtime result provides additional evidence that root detection is not limited to the previously observed Java-level checks and that native code also participates in security-related behavior.

5. Recommendation

- Do Not Rely Solely on Client-Side Root Detection: Client-side root detection is vulnerable to modification via APK reverse engineering, so it should be treated as a defense-in-depth mechanism.
- Perform Transaction Validation on the Backend: Sensitive values like PINs, amounts, and tokens must not rely on client-side validation; all authorization must be processed by a trusted server.

- Protect Token-Generation Mechanisms: Prevent attackers from reproducing tokens through the APK by avoiding hardcoded cryptographic keys or sensitive secrets on the client side.

6. Conclusion

The R2Pay v1.0 Android application was analyzed using static analysis, Smali analysis, runtime Logcat analysis, APK modification, rebuilding, signing, and dynamic testing. The assessment demonstrated that:

- Root detection was identified in MainActivity.onCreate().
- Runtime Logcat confirmed RootBeer detection of /system/bin/su and /system/xbin/su.
- The detected root condition resulted in an intentional divide-by-zero exception that prevented application startup.
- The startup root-detection mechanism was successfully bypassed through Smali modification.
- A second RootBeer-based security check was identified in the token-generation flow.
- PIN validation and formatting logic were identified.
- Amount validation and formatting logic were identified.
- The PIN and amount processing flow leading to token generation was identified.
- The token-generation function and output-processing flow were identified.

The most important conclusion is that client-side security mechanisms can provide useful defense-in-depth, but they should not be treated as trusted security boundaries. Security-sensitive transaction authorization, validation, and cryptographic decisions should ultimately be enforced by a trusted backend.